WEEK 6 -- SQL: Setup and SELECT Basics

Setup: *Use operators and missions from 01_setup.sql. Today is 2026-10-01; timestamps are UTC and costs are PLN. Completed costs use started_at and status='completed'. Do not use window functions.*

1. In-Class Exercises

1.1. IC6.1 -- Operators with recent completed missions

List operators who have at least one completed mission started in September 2026 and whose deleted_at is NULL. Return id, name, and country once per operator, ordered by id. Use a WHERE subquery rather than a JOIN; guest missions must not create operator rows.

1.2. IC6.2 -- Five missions for planned-cost review

Find the five highest-cost missions started before 2026-10-01 whose status is queued or SQL NULL and whose recorded cost is at least 100 PLN. Return mission id, cost, and operator name using a scalar correlated subquery. Retain guest missions with NULL names; break cost ties by smaller mission id.

1.3. IC6.3 -- Operator contact availability

Return every operator's id, original email, and contact_state: null_email for SQL NULL, blank_email for text empty after trimming ASCII spaces, and available otherwise. Keep the original email unchanged. Order by state in that priority, then id; explain why email = NULL cannot identify operators with missing contact details.

1.4. IC6.4 -- Country coverage for a robot training session

A robotics training report must include operators whose country is PL and those whose country is unknown. Return id, name, and country, with PL first and unknown countries last, then id. Explain why country IS NOT DISTINCT FROM 'PL' alone does not include unknown countries, and write the correct filter.

1.5. IC6.5 -- Literal characters in robot model names

Find robot models whose name contains a literal underscore or a literal percent sign. Return id, name, and current unit_cost, ordered by id. Use LIKE with an explicit escape character rather than a regular expression; explain how leaving those characters unescaped would change which model names match.

1.6. IC6.6 -- Prioritize the mission review list

Show all missions started before 2026-10-01, returning id, status, started_at, and cost. Put queued missions first, unknown statuses second, completed third, and aborted last. Within each group put the oldest start first, then the largest cost, then the smallest id; preserve SQL NULL statuses in the output.

1.7. IC6.7 -- Age of completed mission records

Find completed missions started strictly more than 60 elapsed days before TIMESTAMP '2026-10-01 00:00:00'. Return id, started_at, and age_days as complete 24-hour periods. A mission started exactly 60 days earlier is excluded; sort by started_at and id. Do not use the computer's current date or time.

1.8. IC6.8 -- Registration to first completed mission

For operators registered in 2026 before 2026-10-01, return id, name, registered_at, and the earliest started_at of a completed mission before that cutoff. Also show the difference between its UTC calendar date and the registration date in days. Keep operators without completed missions with NULL results; use a correlated subquery and order by id.

1.9. IC6.9 -- Mission history without an abort

Return operators who started at least one mission before 2026-10-01 but have no aborted mission started before that cutoff. Unknown statuses do not count as aborted. Show id and name once, ordered by id, using EXISTS and NOT EXISTS rather than JOINs; operators with no missions must not qualify.

1.10. IC6.10 -- Mission costs above an operator's baseline

Find registered operators' completed September missions whose cost is strictly above their own mean completed-mission cost before 2026-09-01. Require at least two earlier completed missions. Return mission id, operator_id, cost, and baseline_mean rounded to two decimals, ordered by id; compare unrounded values and use correlated subqueries without JOINs.

2. Homework Exercises

2.1. HW6.1 -- Explain operator email-quality issues

Normalize operator email by lowercasing, trimming ASCII spaces, and treating empty text as NULL. Return problematic operators' id, original email, and every applicable reason in the ordered text array below. Apply format reasons only to nonmissing normalized text; order rows by id.

- Reason order: null_email, blank_email, contains_space, missing_at, multiple_at, edge_dot, consecutive_dots.
- The space rule checks original text; blank_email excludes original NULL. Format rules mean zero @, at least two @, a leading/trailing dot, or '..', respectively.

2.2. HW6.2 -- The second mission-cost tier by country

For completed missions started before 2026-10-01, find every mission at its country's second-highest distinct cost. Get country through a scalar operator subquery; guests and unknown countries share one NULL group. Keep all ties and omit countries with fewer than two distinct costs. Return country, mission id, and cost, ordered by country NULLS LAST then id, without JOINs or windows.

2.3. HW6.3 -- An operator experience lifecycle

For operators registered before 2026-10-01, return id, last_completed_at, and the first matching segment below. last_completed_at is the latest start of a completed mission before the cutoff. Use CASE and correlated subqueries, keep dormant reachable, and order by id.

- deactivated: deleted_at before cutoff; new: registration in September; dormant: last_completed_at before cutoff minus 180 days; experienced: any qualifying completed mission; unproven: otherwise.

2.4. HW6.4 -- Mask operator contact details

For operator names of at least two characters, keep the first and last character and replace each middle character with '*'. Lowercase and trim email; reveal only its first local-part character, '***@', and its domain. Return NULL masked_email unless there is exactly one @, nonempty local/domain parts, and no ASCII spaces. Show id, masked_name, masked_email, country, ordered by id.

2.5. HW6.5 -- A mission's share of completed costs

For registered operators' completed September missions, show mission id, operator_id, cost, and that mission's percentage of the operator's total completed-mission cost before 2026-10-01. Compute the denominator with a scalar subquery and round the percentage to two decimals. A zero denominator produces SQL NULL; exclude guests and order by operator_id, then mission id.

2.6. HW6.6 -- Robot models without a completed deployment

List robot models never used in a completed mission started before 2026-10-01. Queued and aborted missions do not count, while completed guest missions do. Return model id, name, and current unit_cost, ordered by id. Use nested subqueries with NOT EXISTS and avoid JOINs; a model may appear in mission_robots yet still qualify.

2.7. HW6.7 -- Operators with a high abort share

Find operators with at least three September missions and an aborted-mission share of at least 25 percent. Every September mission, including unknown status, belongs in the denominator; only aborted rows belong in the numerator. Return id, total_missions, aborted_missions, and abort_pct rounded to two decimals, ordered by exact share descending then id. Use scalar correlated counts without JOINs.

2.8. HW6.8 -- Completed-mission cost ranges

Return every operator's id, completed_mission_count, minimum_completed_cost, maximum_completed_cost, and cost_range for completed missions started before 2026-10-01. Use scalar correlated subqueries rather than JOINs. Operators without completed missions have count zero and NULL for all three cost measures; one completed mission gives range zero. Order by cost_range descending NULLS LAST, then id.

2.9. HW6.9 -- Gaps in completed-mission activity

Generate all 30 calendar dates in September 2026 and return dates with no completed missions started that day. A date with only queued or aborted missions qualifies, as does a date with no missions at all. Use generate_series and NOT EXISTS with inclusive start and exclusive next-day timestamp boundaries. Return activity_day in ascending order.

2.10. HW6.10 -- A seven-day operator activation report

For September 24-30, show activity_day, new_operators, first_completion_operators, and completed_cost_pln. Count registrations that day; an operator's first completion day means the start date of their earliest completed mission, considering all earlier history. Guests contribute completed costs but never operator counts. Preserve all seven dates with zeros, use scalar subqueries without JOINs, and order by date.